

[LAPORAN TEKNIS AKADEMIK]

ANALISIS BACKEND SERVICES, ARSITEKTUR MVC & KEAMANAN DATA

REST API EXPRESS.JS, SUPABASE DATABASE INTEGRATION, DAN AUDIT KEAMANAN

IDENTITAS MAHASISWA & MATA KULIAH

Mata Kuliah	: Pemrograman Web (Teori & Praktikum)
Nama Project	: Sistem Informasi Pramuka SMPN 2 Katapang
Bagian Laporan	: Analisis Backend & Keamanan (PDF)
Dosen Penguji	: Tim Penguji Web Programming
Waktu Analisis	: 8 Juli 2026

PROGRAM STUDI INFORMATIKA / ILMU KOMPUTER
FAKULTAS TEKNOLOGI INFORMASI - 2026

1. Spesifikasi Teknologi & Struktur Backend

Backend pada project Sistem Informasi Pramuka SMPN 2 Katapang dibangun menggunakan runtime environment **Node.js** dengan framework **Express.js** untuk menyediakan layanan RESTful API. Layanan ini terintegrasi dengan database **PostgreSQL Supabase** melalui Supabase Client SDK, serta bertindak sebagai gerbang data bagi modul Client Android dan sinkronisasi eksternal.

Struktur Direktori Backend (Modular Pattern):

Kode sumber backend ditata secara modular berdasarkan tanggung jawab komponen (Separation of Concerns) dalam direktori `backend/src/`:

- `server.js`: File entry-point untuk menjalankan server HTTP pada Port tertentu.
- `app.js`: Konfigurasi global Express, registrasi CORS, Helmet security headers, global error handler, dan pemetaan rute API.
- `config/`: Konfigurasi inisialisasi Supabase Client SDK (`supabase.js` dan `supabaseAndroid.js`).
- `middleware/`: Filter keamanan seperti otorisasi JWT (`authMiddleware.js`), upload file statis (`uploadMiddleware.js`), dan validation formatter (`validatorMiddleware.js`).
- `routes/`: Definisi mapping rute/endpoints dan metode HTTP per modul.
- `controllers/`: Implementasi logika bisnis utama dan interaksi query data ke Supabase.
- `services/`: Utilitas pendukung eksternal seperti pembacaan bucket storage Supabase (`storageService.js`).
- `validations/`: Definisi skema validasi data request body menggunakan `express-validator` .
- `utils/`: Helper kecil untuk pemformatan response JSON standar (`responseHelper.js`) dan rumus Haversine GPS (`gpsHelper.js`).

2. Pemetaan Endpoints REST API

Backend menyediakan endpoint API lengkap yang melayani manipulasi data (CRUD) dan fungsi khusus presensi. Berikut adalah rincian endpoint API penting yang ditemukan di dalam project:

HTTP METHOD	ENDPOINT URL	OTORISASI	DESKRIPSI & AKSI CONTROLLER
POST	/api/auth/login	Public	Autentikasi login pengguna. Memeriksa email, membandingkan hash password, dan menghasilkan JWT token.
GET	/api/auth/profile	Token JWT	Mengambil data profil lengkap user yang aktif beserta detail Siswa/Pembina terkait.
GET	/api/attendance/current	Siswa/Admin	Mengambil agenda absensi GPS aktif (status = 'aktif') untuk check-in.
POST	/api/attendance/checkin	Siswa (Self)	Melakukan absensi geofence GPS + unggah foto selfie. Memvalidasi jarak dan mengunggah gambar ke storage.
POST	/api/attendance/permit	Siswa (Self)	Mengirimkan surat izin/sakit (dokumen PDF/gambar surat dokter) bila berada di luar geofence.
GET	/api/android-absensi	Pembina/Admin	Mengambil log kehadiran terformat yang dikonsumsi oleh aplikasi mobile Android.
GET	/api/kegiatan	Public	Mengambil daftar seluruh kegiatan pramuka publik.
POST	/api/kegiatan	Pembina/Admin	Membuat kegiatan baru beserta dokumentasi gambar.
PUT	/api/kegiatan/:id	Pembina/Admin	Memperbarui informasi kegiatan berdasarkan ID dan mengganti file gambar di storage.
DELETE	/api/kegiatan/:id	Pembina/Admin	Menghapus kegiatan serta menghapus file gambar terkait di bucket storage.

3. Alur Autentikasi dan Otorisasi (Auth Flow)

Keamanan routes diatur menggunakan otentikasi token berbasis **JSON Web Token (JWT)** dan otorisasi berbasis Peran (Role-Based Access Control - RBAC).

A. Alur Autentikasi (JWT Token Generation & Validation):

1. Client mengirim request POST ke `/api/auth/login` dengan body JSON berisi `email` dan `password`.
2. Controller mencari user di database Supabase berdasarkan email. Jika ditemukan, password di-dekripsi dan dicocokkan menggunakan `bcrypt.compare()`.
3. Jika valid, JWT token di-generate di server menggunakan payload: `{ id, email, role }` dan ditandatangani dengan `process.env.JWT_SECRET`, dengan masa kadaluarsa 24 jam.
4. Token dikembalikan ke client dalam format JSON response.
5. Untuk setiap request ke endpoint terproteksi, client wajib menyertakan header `Authorization: Bearer <token>`.

B. Middleware Validasi Otorisasi (RBAC):

File `backend/src/middleware/authMiddleware.js` menyediakan middleware penyeleksi akses:

- **verifyToken:** Mengekstrak substring setelah kata 'Bearer ' pada header `Authorization`, memverifikasi kecocokan tanda tangan token menggunakan `jwt.verify`, mengambil profil pengguna terbaru dari database, lalu memasang objek pengguna ke properti `req.user`.
- **verifyAdmin:** Memastikan `req.user.role === 'admin'`. Jika tidak sesuai, mengembalikan response error 403 Forbidden.
- **verifyPembina:** Memastikan pengguna memiliki peran `'pembina'` atau `'admin'`.
- **verifySiswa:** Memastikan pengguna memiliki peran `'siswa'` atau `'admin'`.

4. Analisis Implementasi CRUD

Sebagai perwakilan implementasi pola CRUD (Create, Read, Update, Delete) pada backend, modul **Kegiatan** dianalisis secara mendalam. Modul ini melibatkan file data input, filter, validator, controller, dan storage bucket:

A. Modul File yang Digunakan:

- Router: `backend/src/routes/kegiatanRoutes.js`
- Controller: `backend/src/controllers/kegiatanController.js`
- Validation Schema: `backend/src/validations/kegiatanValidation.js`
- Supabase Config: `backend/src/config/supabase.js`
- Storage Service: `backend/src/services/storageService.js`

B. Rincian Alur Kerja CRUD:

1. **Create Kegiatan (POST `/api/kegiatan`)**: Client mengirimkan request multipart/form-data (karena menyertakan file upload gambar). Request ditangkap oleh `upload.single('gambar')` (menggunakan Multer) -> dilewatkan ke skema `createKegiatanValidation` untuk memastikan nama kegiatan dan tanggal (Format DATE) terisi -> jika lolos, controller `createKegiatan` akan mengunggah gambar ke Supabase Storage Bucket 'kegiatan' melalui `uploadFile` -> mendapatkan URL publik gambar -> menyimpan seluruh record ke tabel `kegiatan` -> mengembalikan data JSON baru dengan status HTTP 201 Created.
2. **Read Kegiatan (GET `/api/kegiatan` & GET `/api/kegiatan/:id`)**: Endpoint ini bersifat publik (bebas token). Controller mengeksekusi select query ke Supabase `.from('kegiatan').select('*').order('tanggal', { ascending: false })` -> Mengembalikan array JSON berisi seluruh kegiatan untuk dirender di landing page.
3. **Update Kegiatan (PUT `/api/kegiatan/:id`)**: Menerima ID dari parameter URL. Jika ada upload berkas gambar baru, gambar lama dihapus dari storage menggunakan helper `deleteFile` dan digantikan gambar baru -> Melakukan partial update (hanya kolom yang dikirim di request body) -> Mengembalikan data kegiatan ter-update.
4. **Delete Kegiatan (DELETE `/api/kegiatan/:id`)**: Menerima ID dari parameter URL. Mencari data kegiatan di database -> menghapus file gambar di storage bucket -> menghapus baris data di tabel `kegiatan` -> mengembalikan response sukses.

5. Analisis Pola Arsitektur MVC & Next.js App Router

A. Implementasi MVC pada Backend Express.js

Backend Express menerapkan variasi arsitektur **Model-View-Controller (MVC)** tradisional:

- **Model (Data Layer):** Diwakili oleh skema database di PostgreSQL Supabase dan konfigurasi koneksi SDK di `backend/src/config/supabase.js`. Representasi data tidak menggunakan berkas kelas OOP (ORM), melainkan manipulasi baris objek JSON relasional secara dinamis.
- **View (Presentation Layer):** Karena backend bertindak sebagai Headless REST API, View dipisahkan secara fisik. View dikembalikan dalam format **JSON standard response** (menggunakan `responseHelper.js`) yang dikonsumsi oleh web frontend Next.js atau client mobile Android untuk dirender secara visual.
- **Controller (Business Logic Layer):** Berkas di folder `backend/src/controllers/` bertindak sebagai Controller penuh. Berkas ini menerima input request HTTP, berinteraksi dengan Model (Supabase), mengolah data, menerapkan validasi sanitasi, dan merumuskan respons JSON.

B. Pengganti Konsep MVC pada Frontend Next.js App Router

Di bagian frontend, framework Next.js 16 App Router menggantikan pola MVC tradisional dengan arsitektur modern berbasis **React Server Components (RSC)** dan **Server Actions**:

- **View:** Diwakili oleh komponen halaman `page.tsx` dan komponen UI modular (JSX/TSX). UI ini dirender di server-side menjadi HTML statis untuk pengiriman cepat ke client.
- **Model & Controller (Server Actions):** Berkas di folder `src/app/actions/` menggabungkan peran Model dan Controller. Fungsi Server Actions bertindak sebagai Controller yang dipanggil langsung dari form submit (View). Fungsi-fungsi ini berinteraksi langsung ke database Supabase (Model) menggunakan SDK server-side, mengeksekusi logika, memvalidasi sesi, lalu memerintahkan View untuk me-revalidate cache (re-render) halaman secara dinamis melalui `revalidatePath`.

6. Audit & Analisis Keamanan Data (Security Audit)

Dilakukan evaluasi mendalam terhadap aspek keamanan data pada sistem backend dan frontend:

PARAMETER KEAMANAN	KELEBIHAN / IMPLEMENTASI SAAT INI	KEKURANGAN & KERENTANAN YANG DITEMUKAN
Authentication	Menggunakan JWT token yang ditandatangani secara kriptografis dengan algoritma standard HS256.	Masa berlaku token cukup lama (24 jam) dan tidak ada mekanisme rotasi token (refresh token). Jika token dicuri, attacker memiliki akses penuh selama 24 jam.
Authorization	Penerapan RBAC yang ketat di tingkat middleware rute API dan validasi role internal pada Server Actions. Mencegah kerentanan IDOR pada check-in.	Validasi otorisasi di Android controller hanya mengecek peran Pembina/Admin secara umum, tidak membatasi kepemilikan data spesifik di beberapa endpoint.
Password Handling	Password disimpan dalam bentuk hash searah menggunakan library <code>bcryptjs</code> (salt-rounds 10). Kini dilengkapi validasi kekuatan kata sandi ketat (min 8 karakter, kombinasi huruf besar, huruf kecil, angka, & simbol).	- (Kerentanan password lemah telah diatasi melalui integrasi regex validator baru).
Session Management	Frontend menyimpan JWT di dalam HttpOnly Cookie . Di sisi API Backend , token JWT kini diamankan dengan sistem blacklist memori saat logout untuk mencabut validitas token lama .	Penyimpanan blacklist saat ini bersifat <i>in-memory Set</i> (akan tereset jika server restart), yang ideal untuk single-instance server namun memerlukan Redis jika di-scale up ke multi-instance.
Input Validation	Validasi ketat tipe data, keharusan field (NOT NULL), dan format email/tanggal diatur menggunakan <code>express-validator</code> sebelum masuk controller database.	Beberapa endpoint PUT (update) tidak membatasi ukuran input teks panjang secara eksplisit di validator, yang berpotensi memicu Payload Too Large / DoS.
XSS & CSRF Protection	Helmet middleware diaktifkan untuk mengatur Content Security Policy (CSP), mematikan MIME sniffing, dan mengaktifkan XSS auditor. Next.js actions secara native melindungi dari serangan CSRF.	CORS policy di Express backend diatur membolehkan origin wildcard (*) jika variabel env tidak diatur dengan benar, berpotensi memicu Cross-Origin Resource Sharing vulnerability.

Rekomendasi Perbaikan Sistem Keamanan:

STATUS IMPLEMENTASI KEAMANAN:

- 1. Token Blacklisting:** TERPASANG Telah diimplementasikan menggunakan modul `tokenBlacklist.js` pada middleware `verifyToken` dan controller `logout`.
- 2. Validasi Password Kuat:** TERPASANG Telah ditambahkan validasi regex di backend `userValidation.js` dan frontend server action `users.ts`.
- 3. Mitigasi Fake GPS (Velocity Check):** TERPASANG Telah dipasang fungsi audit anomali kecepatan perpindahan geografis siswa (maks 120 km/jam) pada controller `absensiController.js` dan `attendanceController.js` untuk mendeteksi koordinat palsu.

- 1. Migrasi Blacklist ke Redis (Untuk Multi-Instance):** Mengubah sistem *in-memory Set* yang baru dipasang menjadi database Redis agar pemblokiran token sinkron di seluruh instance server cluster.
- 2. Rotasi Token dengan Refresh Token:** Kurangi masa kadaluarsa Access Token JWT menjadi 15 menit, dan terbitkan Refresh Token (HttpOnly Cookie) berumur 7 hari untuk memperbarui Access Token secara otomatis.
- 3. Perkuat Proteksi Geofencing dari Fake GPS di Sisi Mobile Client:** Setelah memasang sistem *Velocity Anomaly Check* di sisi backend, direkomendasikan untuk menambahkan deteksi status Mock Location di client Android (menggunakan library android location API `isFromMockProvider()`) sebelum koordinat dikirimkan ke server demi proteksi lapis kedua.

Laporan Ujian Web Programming - Analisis Backend & Keamanan Halaman 1 dari 1